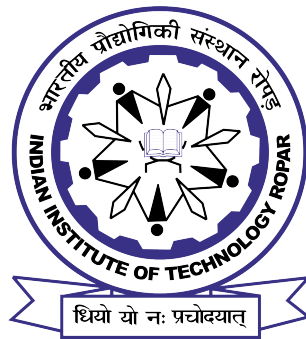


SDN Based Deterministic Time Sensitive Networking

A
Thesis Submitted
in Partial Fulfillment of the Requirements
for the Degree of
MASTER OF TECHNOLOGY
By

PALLAV KUMAR
Under the Supervision of
Dr. ABHINANDAN SP



Department of Computer Science & Engineering
Indian Institute of Technology Ropar
May 8, 2025

DECLARATION

This is to certify that the thesis entitled “**SDN Based Deterministic Time Sensitive Networking**”, submitted by me to the *Indian Institute of Technology Ropar*, for the award of the degree of Master of Technology, is a bonafide work carried out by me under the supervision of Dr. Abhinandan SP. The content of this thesis, in full or in parts, have not been submitted to any other University or Institute for the award of any degree or diploma. I also wish to state that to the best of my knowledge and understanding nothing in this report amounts to plagiarism.

Sign: _____

Pallav Kumar
Department of Computer Science & Engineering,
Indian Institute of Technology Ropar,
Rupnagar-140001, Punjab, India.

Date: _____

CERTIFICATE

This is to certify that the thesis entitled “**SDN Based Deterministic Time Sensitive Networking**”, submitted by Pallav Kumar (2023CSM1010), a master’s student in the *Department of Computer Science & Engineering, Indian Institute of Technology Ropar*, for the award of the degree of Master of Technology, is a record of an original research work carried out by the candidate under my supervision and guidance. The thesis has fulfilled all requirements as per the regulations of the institute and in my opinion has reached the standard worthy of the award of the degree. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Sign: _____

Supervisor: Dr. Abhinandan SP
Department of Computer Science & Engineering,
Indian Institute of Technology Ropar,
Rupnagar-140001, Punjab, India.

Date: _____

ACKNOWLEDGEMENTS

My heartfelt gratitude goes out to several individuals whose unwavering support and guidance have been instrumental in the completion of this thesis. I take this opportunity to acknowledge and express my gratitude to all those who supported and guided me during my thesis. I am deeply grateful to my supervisor **Dr. Abhinandan SP**, for giving me the opportunity to work under his direction and explore a significant problem statement in the real world. I will always be grateful for his unwavering support and encouragement, whether related to this thesis or beyond. I also thank the respected Head of the Department and all the faculty members of the Department of CSE.

Dr. Abhinandan's profound expertise not only illuminated complex issues but also played a pivotal role in overcoming significant challenges encountered during the research process. His depth of knowledge brought clarity to intricate problems, offering invaluable insights that propelled the project forward.

Moreover, I extend my sincere appreciation to my colleague, PhD scholar Anjani, whose collaborative spirit, willingness to share expertise, and insightful contributions during brainstorming sessions have enriched this endeavor immeasurably. Anjani's support, both technically and morally, has been a constant source of encouragement, fostering a collaborative and conducive environment for research and learning.

I extend my heartfelt gratitude and respect to those who have guided me to this point and contributed to all that I have achieved, especially my parents. Their unwavering support during my struggles and difficult times has made this journey possible. Their guidance and encouragement have not only shaped the outcome of this thesis but have also contributed significantly to my personal and professional growth. Last but not least, I would like to thank myself—for believing in my potential and staying resilient through every challenge faced along this journey.

Sincerely
Pallav Kumar

ABSTRACT

Modern IP networks are naturally best-effort in nature, causing unpredictable delay and jitter, which is not acceptable for applications which require deterministic performance. To meet this challenge the concept for Deterministic Networking (DetNet) has come into the picture which promises the bounded latency and minimal jitter throughout the large-scale networks. In this work I implemented the main components of the research paper titled “Towards Large-Scale Deterministic IP Networks”[1], which is mainly focusing on the scalable and efficient deterministic scheduling in IP-based data networks. My implementation uses Column Generation with Randomized Rounding (CGRR) approach, as proposed in the paper to calculate the bounded delay for multi-commodity flows. I developed a full simulation pipeline using Python and NetworkX, which incorporates topology parsing, demand specification, delay-aware path computation, LP formulation, column generation and finally integer flow assignments. My implementation shows the practicality of the proposed method in real-world network scenarios which bridges the gap between theory and deterministic IP networking solutions.

Keywords: Deterministic Networks, IP Networking, Multi-commodity Flows, Column Generation, Network Scheduling, NetworkX, Python

Contents

List of Figures	v
List of Algorithms	vii
1 Introduction	1
2 Background and Related Work	5
2.1 The Need for Deterministic Networking	5
2.2 Time-Sensitive Networking and DetNet	6
2.3 Delay-Constrained Multi-Commodity Flows	6
2.4 Column Generation with Randomized Rounding (CGRR)	7
2.5 Related Work	8
2.6 Simulation-Oriented Implementations	8
3 System Architecture and Design	11
3.1 High-Level Architecture	11
3.2 Module-Level Design	12
3.3 Configuration and Flow Control	17
3.4 Cycle-Based Scheduling	18
3.5 Output Format and Logging	19
3.6 Summary	20
4 Implementation Details	21
4.1 Flow Modeling and Packet Abstraction	21
4.2 Network Representation	22
4.3 Flow Admission using CGRR	22
4.3.1 Column Generation	23
4.3.2 Randomized Rounding	23
4.4 Node Implementation and Packet Flow	23
4.5 Cycle-Based Simulation Engine	24
4.6 Random Flow Generation	24
4.7 Output Logging and JSON Reporting	25
4.8 Summary	25
5 Conclusion and Future Work	27
5.1 Contributions	28
5.2 Future Work	29

Bibliography

31

List of Figures

1.1	A simple DetNet-enabled network [2]	2
3.1	High level overview of project architecture	12
3.2	JSON configuration structure	17
3.3	JSON Output Structure	19

List of Algorithms

1	Packet Class Definition	13
2	Flow Class Definition	13
3	Network Class Definition	14
4	CoreNode Class Definition	15
5	IngressNode Class Definition	16

Chapter 1

Introduction

THE Internet Protocol (IP) has long served as a basic communication method across the world's network, which enables the rapid growth of the web, cloud computing, streaming services and countless other technologies. Scalability, flexibility and simplicity of the IP-based networks are the main reasons for this success. However, the main defining feature of traditional IP networks is its *best-effort delivery model*-a method where network does not guarantee delivery time, reliability or bandwidth. Although the best-effort networking is suitable for non-critical applications such as e-mail or video streaming (where some loss or delay is tolerable), it fails to meet the strict requirements of **mission-critical applications** such as industrial control systems, autonomous vehicles, financial trading platforms, telemedicine, and real-time immersive systems. These applications require a **deterministic guarantees** from the underlying network: low and bound latency, minimal jitter, zero package loss and guaranteed bandwidth. For example, an autonomous vehicle responding to the traffic signal requires communication delays within milliseconds to ensure safety of passenger. Similarly in industrial automation, failure to meet real-time deficiency can have dangerous consequences. In such conditions traditional IP networks proves to be less effective.

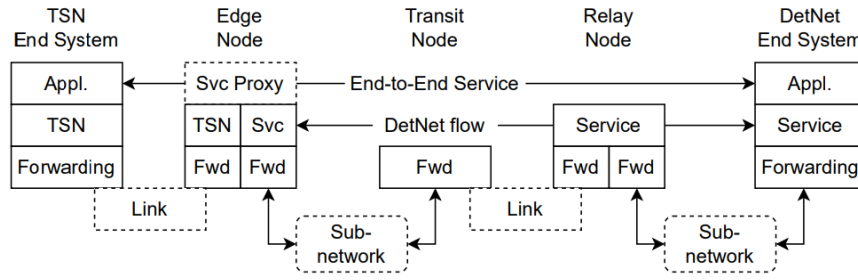


FIGURE 1.1: A simple DetNet-enabled network [2]

The concept of **Deterministic Networking (DetNet)**[2] has been proposed to face this challenge and is actively being standardized by bodies like the IETF[3]. The main goal of DetNet is to provide deterministic performance guarantees to Layer 3 (IP) networks by introducing scheduling, path reservation, and congestion control mechanisms which enables reliable and predictable data transmission across complex topologies. The earlier solutions such as Time-Sensitive Networking (TSN)[4] focused on Layer 2 (Ethernet) networks and DetNet builds on the same ideas but adapts them to the IP domain, enabling end-to-end deterministic communication over mixed and large-scale networks.

Despite of promising advances in DetNet theory and standards, **scalability** is still a major obstacle. As networks grow in size and complexity, to ensure that a set of communication demands are met with deterministic guarantees under delay constraints becomes a challenging optimization problem. Traditional resource allocation methods quickly become intractable or inefficient at large scales. Thus, there is a pressing need for scalable, efficient, and implementable algorithms that can support deterministic traffic engineering in large IP networks.

The research paper “**Towards Large-Scale Deterministic IP Networks**”[1] presents a novel solution to this problem by formulating the deterministic flow allocation problem as a **delay-constrained multi-commodity flow** problem. The paper introduces a two-stage algorithm based on **Column Generation and Randomized Rounding (CGRR)** to compute fractional multi-commodity flows using linear programming (LP) and then convert them into integer solutions that can be deployed in real networks. This algorithm maintains a balance between optimality and scalability also handling the inherent complexity of the problem while providing practical solutions that can scale to large topologies with many flows.

This project aims to **implement** the proposed algorithm in a practical simulation environment using Python. I designed and developed a complete implementation pipeline that includes:

- Parsing and representing arbitrary network topologies,
- Defining and handling communication demands with delay and rate requirements,
- Solving the LP-based fractional flow allocation problem using column generation,
- Applying randomized rounding to generate deterministic integral flows,

I used Python libraries such as **NetworkX** for graph modeling and manipulation, and **SciPy** for linear programming. The implementation is modular and extensible which enables the evaluation of different topologies and flow patterns.

My work provides a complete and effective implementation of the theoretical model proposed in the paper which helps in offering insights into the performance, practicality, and limitations of the CGRR approach in real-world networking scenarios.

Contributions of this Project:

- A full Python-based implementation of the research paper titled "**Towards Large-Scale Deterministic IP Networks**"[1].
- An extensible framework for simulating and analyzing deterministic flow allocations on arbitrary network topologies.
- It will be helpful in evaluating and benchmarking algorithm performance in terms of flow acceptance, delay compliance, and runtime.

Chapter 2

Background and Related Work

THIS chapter is going to explore the key concepts and prior work that form the foundation of deterministic networking. We will start with a discussion on the motivation and principles behind deterministic network design, then we will move towards an overview of important technologies such as Time-Sensitive Networking (TSN)[4] and Deterministic Networking (DetNet). Next we will introduce the core optimization problem addressed in this project — the delay-constrained multi-commodity flow problem — and present the CGRR algorithm as a scalable and practical solution. Finally, we summarize related efforts in this space and position our work in the broader context of network simulation.

2.1 The Need for Deterministic Networking

The demand for predictability and bounded latency in communication networks has grown significantly because the digital infrastructure continues to support an increasing number of critical applications. Since traditional IP networks are fundamentally designed under a **best-effort** delivery model that's why it is only sufficient for general data traffic and cannot guarantee the strict delay, jitter, and reliability requirements of **time sensitive applications**.

Deterministic packet delivery is required in many applications such as industrial control systems, autonomous vehicles, smart grids, and virtual/augmented reality systems. The violation of a timing constraint in such systems could lead to catastrophic outcomes such as safety hazards and system failures. This has inspired the need to progress the networking technologies toward Time-Deterministic behavior.

2.2 Time-Sensitive Networking and DetNet

Time-Sensitive Networking (TSN)[4] has been developed by the IEEE to overcome the limitations of best-effort networks. It primarily target the Layer 2 (Ethernet) networks and introduces features like traffic shaping, time-aware scheduling, and resource reservation, which enable precise control over packet delivery times.

Building on the ideas of TSN, the **Deterministic Networking (DetNet)** initiative, led by the IETF, extends these capabilities to Layer 3 (IP) networks. DetNet proposes architectural components and mechanisms to provide:

- Bounded end-to-end delay and jitter
- Zero congestion loss
- Redundancy and disjoint paths for reliability
- Integration with traditional IP routing

However, implementing these guarantees in large-scale, dynamic IP networks introduces significant computational and architectural challenges.

2.3 Delay-Constrained Multi-Commodity Flows

One of the main challenges in deterministic networking is the allocation of network resources in a way that meets **delay constraints** across multiple source-destination pairs, often referred to as **flows**. This is formally modeled as a **delay-constrained multi-commodity flow problem**, which is known to be NP-hard.

In this problem, each flow has:

- A source and destination node
- A required arrival rate (bandwidth)
- A burst size
- A maximum allowed end-to-end delay

The goal is to determine which flows can be admitted and to compute the paths and rate allocations for them such that all constraints are satisfied.

2.4 Column Generation with Randomized Rounding (CGRR)

The paper “Towards Large-Scale Deterministic IP Networks”[1] proposes a scalable two-stage solution to this problem using **Column Generation with Randomized Rounding (CGRR)**:

1. **Column Generation (CG)**: A fractional flow is computed by solving a Linear Programming (LP) formulation. Instead of generating all possible paths beforehand (which is computationally expensive), the CG method incrementally adds only the most beneficial paths, reducing the size of the LP and improving scalability.
2. **Randomized Rounding (RR)**: Once the fractional solution is obtained, it is converted into a feasible integral (deterministic) solution using a randomized rounding technique that preserves feasibility with high probability.

This combination enables practical computation of delay-constrained flows over large topologies without requiring exhaustive enumeration or brute-force search.

2.5 Related Work

Few related research work has been done in the area of deterministic and time-sensitive networking:

- **TSN standards** [4] (IEEE 802.1 series) introduced concepts such as time-aware shapers and scheduled traffic for Layer 2 networks.
- **DetNet architecture** [2] (IETF drafts) outlines a framework for bounded-latency services over IP and MPLS networks.
- Earlier deterministic routing approaches relied on per-flow reservations and centralized scheduling, which are difficult to apply to large-scale networks with dynamic demands.

The CGRR method stands out by balancing theoretical rigor with practical implementability and is particularly well-suited for modern software-defined or programmable networks.

2.6 Simulation-Oriented Implementations

The CGRR approach has been theoretically evaluated but its real-world simulation and system-level behavior had not been demonstrated in public implementations prior to this work. My project bridges this gap by implementing a **time-deterministic network simulator** that:

- Uses **cycle-based scheduling** to simulate time slots for packet transmission.
- Models core and ingress nodes as functional software entities.
- Simulates deterministic packet delivery with queuing and propagation delays.
- Supports flow admission based on CGRR-computed paths and rates.
- Accepts both **static JSON-based configurations** and **randomized flow generation**.

The modular structure of the simulator allows easy extension, evaluation, and integration with real-time networking policies. The system is designed to provide both per-packet visibility and aggregate flow-level statistics, enabling comprehensive performance analysis of deterministic delivery strategies.

Chapter 3

System Architecture and Design

THIS chapter provides a detailed overview of the architecture and design of the time-deterministic IP network simulator developed as part of this project. The simulator uses a cycle-based scheduling mechanism for simulating deterministic packet transmission across a network of nodes. The architecture reflects the conceptual design which is proposed in the paper “*Towards Large-Scale Deterministic IP Networks*” [1] and implements its core algorithmic framework—Column Generation with Randomized Rounding (CGRR)—within a practical, modular software system.

The architecture was created with readability, simulation accuracy, and extensibility in mind. Through a JSON-based input system, it supports network topologies, flow definitions, and cycle timing parameters that can be changed and it also produces structured output detailing admitted flows, scheduling decisions, and delivery status.

3.1 High-Level Architecture

The following are the main layers on which the simulator operates:

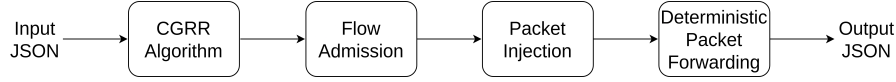


FIGURE 3.1: High level overview of project architecture

- **Input Configuration Layer:** Interprets user-defined network configurations and flow requests from a JSON file.
- **Network Topology Layer:** Represents the graph of nodes and links with characteristics like propagation delay, bandwidth, and queuing delay.
- **Algorithmic Layer:** Implements the CGRR algorithm to determine viable paths and assign rates for deterministic flows.
- **Node Behavior Layer:** Models the timing, forwarding, and packet injection behaviour of ingress and core nodes.
- **Scheduling Layer:** Implements deterministic packet transmission across the network by putting into practice a cycle-based time model.
- **Output Layer:** Logs simulation events, flow statuses, and generates output as a structured JSON file.

3.2 Module-Level Design

The system is implemented using Python and organized across multiple modular files for clarity and ease of maintenance.

`models.py`

This module defines the basic building blocks of the simulation, including:

- **Packet:** Represents individual packets flowing through the network. The code snippet of the packet class definition has been presented in algorithm 1.

Algorithm 1: Packet Class Definition

class Packet:

```

def __init__(self, size, flow_id, label=None):
    self.size ← size ;                      // Size of the packet in KB
    self.flow_id ← flow_id ;                // Identifier for the flow
    self.label ← label ;                    // Cycle label for transmission

```

- **Flow:** Represents a data flow with their respective attributes such as rate, burst size, delay constraints, and path. The code snippet of the flow class definition has been presented in algorithm 2.

Algorithm 2: Flow Class Definition

class Flow:

```

def __init__(self, flow_id, arrival_rate, burst_size, max_e2e_delay,
max_pkt_size, src, dest):
    self.flow_id ← flow_id ;                // Identifier for the flow
    self.arrival_rate ← arrival_rate ;      // Packet arrival rate (e.g., Mbps)
    self.burst_size ← burst_size ;          // Maximum burst size (e.g., KB)
    self.max_e2e_delay ← max_e2e_delay ;    // Max end-to-end delay
                                            (e.g., ms)
    self.max_pkt_size ← max_pkt_size ;      // Maximum packet size (e.g.,
                                            KB)
    self.src ← src ;                        // Source node
    self.dest ← dest ;                      // Destination node
    self.packets ← [ ] ;                    // Queue of packets in the flow

```

- Utility function for generating random flows. This function take an integer as a parameter which is number of random flows user wish to generate randomly. Another utility function is also present in this file to display all the flows.

network.py

This module defines the **Network** class whose code snippet can be seen in algorithm 3 and this module also provides helper function for:

- Manipulating the network topology.
- Adding links with specific delays and bandwidths.
- Calculating the overall delay which occurs in the network, given a specific path in the network and the queuing delays of all the nodes.

The network defined in this module is represented as a graph and for this graph representation and traversal it uses **NetworkX** package of the Python.

Algorithm 3: Network Class Definition

class Network:

```

def __init__(self):
    self.graph ← nx.Graph() ;                      // Graph G(V,E)
    self.delays ← {} ;                             // Delay for each link e ∈ E
    self.bandwidths ← {} ;                         // Bandwidth for each link e ∈ E

```

core_node.py

This module defines the **CoreNode** class that simulates the behavior of the intermediate nodes as discussed by Liu et al. [1]. The code snippet of the definition of the CoreNode class has been presented in algorithm 4. Key responsibilities of this module include:

- Receiving and forwarding packets based on flow schedules and follows the three queue structure as discussed by Liu et al. [1].
- Managing cycle-based delays and forwarding logic and in each cycle it process one queue and each queue is allocated time in the round robin fashion.
- Managing the mapping table at each node which maintains the relation between incoming cycle and the outgoing cycle of the packet.

Algorithm 4: CoreNode Class Definition

class CoreNode:

```

def __init__(self, node_id, cycle_duration_T):
    self.node_id ← node_id
    self.cycle_duration_T ← cycle_duration_T
    self.queues ← {{}}, {}, {} ;                               // Three FIFO queues
    self.current_cycle ← 0
    self.last_cycle_time ← time.time() - (cycle_duration_T / 106)
    self.lock ← threading.Lock()
    self.mapping_table ← {} ;                               // Mapping table: maps (in_port,
        in_label) to (out_port, out_label)
    self.link_delays ← {} ;    // Store delays for each link to neighbors
    self.queuing_delay ← 0.0
    self.active_queue_index ← 0 ;    // Track active queue during each
        cycle
    self.connected_nodes ← {} ;                               // Connect to other nodes
    self.next_hop_map ← {} ;    // Track which node is the next hop for
        each flow
    self.packet_flow_map ← {} ;    // Store the flow ID for each packet
    self.routing_table ← {} ; // Routing table: maps flow_id to next hop

```

ingress_node.py

This module defines the **IngressNode** class which simulates the behavior of the ingress node as discussed by Liu et al. [1]. The code snippet of the definition of the IngressNode class has been presented in algorithm 5. This module is responsible for:

- Entry of packets into the network and the amount of packet to be entered is based on the shaping parameter of the flow.
- Applying traffic shaping, based on the shaping parameter calculated and timing controls.
- Triggering flow transmission according to scheduling cycles.

IngressNode class is child class of the **CoreNode** class. Egress functionality is handled within this module when the destination is reached.

Algorithm 5: IngressNode Class Definition

```
class IngressNode(CoreNode):
    def __init__(self, cycle_duration_T, node_id=1):
        super().__init__(node_id, cycle_duration_T)
        self.flow_queues ← {} ;                // Queues for each flow
        self.flow_order ← deque() ;            // Order of flows
        self.flow_paths ← {} ;                // Path for each flow
        self.flow_completed_callback ← None ; // Callback on flow completion
```

algorithms.py

This module implements different utility functions used by the control plane. It also implements Column Generation and Randomized Rounding algorithm (CGRR), which is the core algorithm of the control plane as discussed by Liu et al. [1]. The main functionality of this module are:

- Computes fractional multi-commodity flows with delay constraints using linear programming.
- Uses column generation to dynamically introduce paths that improve flow feasibility.
- Applies randomized rounding to obtain integral (deterministic) paths.
- Determines which flows can be admitted into the simulation.

This module serves as the decision engine for flow acceptance and path computation.

main.py

This module is the entry point of the simulation. The main responsibilities of this module include:

- Parsing command-line arguments and input JSON.
- Displaying all the flows.
- Initializing the network topology and nodes.
- Triggering flow admission via CGRR.
- Running the cycle-based simulation loop.
- Generating and saving the output JSON file.

3.3 Configuration and Flow Control

```
```json
{
 "simulation_parameters": {
 "cycle_duration_T": 10
 },
 "network": {
 "nodes": [1, 2, 3, 4],
 "links": [
 {
 "node1": 1,
 "node2": 2,
 "delay": 1.0,
 "bandwidth": 100
 },
 ...
],
 "queuing_delays": {
 "1": 0.5,
 "2": 1.0,
 "3": 0.8,
 "4": 0.7
 }
 },
 "flows": [
 {
 "flow_id": 1,
 "arrival_rate": 10,
 "burst_size": 7.5,
 "max_e2e_delay": 50,
 "max_pkt_size": 1.5,
 "src": 1,
 "dest": 4
 },
 ...
]
}
```
```

FIGURE 3.2: JSON configuration structure

There are two modes of operation supported by this simulator:

1. **Predefined flows:** In this mode of operation the user has to create a JSON configuration file which contains the network information and flow information on which the simulation work. By default, the program loads a JSON configuration file named `network_config.json` but user can also load custom JSON file using specific command while running. The structure of the JSON configuration file can be seen in fig 3.2.
2. **Random flows:** This is the second mode of operation where the user has to pass the number of random flows that they need. By default, the first mode of operation is performed by the simulator, but user can use this mode of operation using specific command while running. The random flows are generated using `"generate_random_flows"` function of the `"models.py"` module discussed in 3.2.

Each input flow of the program has the following characteristics:

1. Source and destination nodes.
2. Arrival rate.
3. Burst size.
4. Maximum end-to-end delay required for that specific flow.
5. Maximum packet size in that flow.

Cycle duration can also be configured, which allows simulation for different timing granularities.

3.4 Cycle-Based Scheduling

The main feature of this simulator is its **cycle-based packet scheduling**. Time is split into fixed-duration cycles (e.g. 10 microseconds). During each cycle:

- Packets enter into the network through ingress nodes according to the shaping rules.

- Core nodes forward packets and maintain the quasi-static mapping table which maps the input cycle of the packet to the output cycle of the packet.
- Queuing and propagation delays are simulated to determine actual delivery times.

This design approximates the time-triggered behavior of the architecture discussed by Liu et al. [1].

3.5 Output Format and Logging

```

```json
{
 "simulation_parameters": {
 "cycle_duration_T": 10
 },
 "network": {
 "nodes": [1, 2, 3, 4],
 "links": [...],
 "queuing_delays": {...}
 },
 "flows": [
 {
 "flow_id": 1,
 "arrival_rate": 10,
 "burst_size": 7.5,
 "max_e2e_delay": 50,
 "max_pkt_size": 1.5,
 "src": 1,
 "dest": 4,
 "admitted": true,
 "path": [1, 2, 4],
 "shaping_parameter": 3.75
 },
 ...
],
 "admitted_flows_count": 2,
 "total_flows_count": 3,
 "simulation_complete": true,
 "completion_status": {
 "1": true,
 "2": true
 }
}
```

```

FIGURE 3.3: JSON Output Structure

Simulation results are being saved in a structured JSON file which contains:

- Status of flow admission.
- Paths that are chosen.

- Timing constraints.
- Status of packet completion.
- Status of the final simulation.

This output can be used for validation, visualization, and further analysis. The structure of the output JSON file can be seen in fig 3.3.

3.6 Summary

The simulator provides a comprehensive environment for modeling deterministic IP networks with a focus on implementation fidelity. By isolating key components such as topology, node behavior, and scheduling logic into modular files, the architecture remains clear and extensible. This design mirrors the principles laid out in the original research paper while adapting them into a practical, testable framework.

Chapter 4

Implementation Details

THIS chapter provides a deeper insight into the internal implementation of the deterministic network simulator, highlighting the algorithms, data structures, simulation mechanics, and configuration options developed to support a faithful translation of the concepts from “Towards Large-Scale Deterministic IP Networks” [1] into working code. The primary objective of this implementation was to realize a modular and flexible simulation platform capable of modeling time-deterministic packet delivery using the Column Generation with Randomized Rounding (CGRR) algorithm.

The implementation is written in **Python** and leverages several well-established libraries such as **NetworkX** for graph representation, **SciPy** for linear programming, and Python’s built-in JSON tools for structured configuration and output. The simulation operates in discrete time steps called cycles, during which packets are injected, forwarded, and tracked according to precomputed schedules.

4.1 Flow Modeling and Packet Abstraction

At the core of the simulation are two fundamental data structures: Flow and Packet. Each Flow object encapsulates parameters including source and destination nodes, arrival

rate (in Mbps), burst size (in KB), maximum end-to-end delay (in milliseconds), and maximum packet size. These parameters directly map to the flow admission model defined in the research paper [1].

The `Packet` class models individual packet instances that are generated at the ingress node and propagated hop-by-hop through the network. Each packet carries metadata including the flow ID, current position, timestamp of injection, and scheduled time of delivery. The shaping and queuing behavior of these packets is governed by cycle-aligned schedules computed during the flow admission phase.

4.2 Network Representation

The network is implemented as an undirected graph, constructed using `NetworkX`. Nodes in the graph represent routers or switches, and edges represent bidirectional links with specified propagation delays and bandwidths. The topology is configured via a JSON file, which specifies:

- A list of nodes (represented by numeric or string IDs),
- Links with attributes: `node1`, `node2`, `delay`, and `bandwidth`,
- Node-specific queuing delays, stored in a dictionary keyed by node ID.

The `network.py` module parses this configuration and builds the graph, also offering utility functions for querying delay along paths, computing shortest paths, and managing topology-related metadata.

4.3 Flow Admission using CGRR

The flow admission logic is implemented in the `algorithms.py` module, following the two-stage CGRR approach.

4.3.1 Column Generation

Column generation is employed to iteratively solve a linear programming relaxation of the multi-commodity flow problem. Rather than generating all possible paths upfront, the algorithm incrementally identifies paths that contribute to minimizing delay while satisfying flow constraints. In each iteration, the algorithm solves a restricted LP and then introduces new columns (paths) that improve the objective.

4.3.2 Randomized Rounding

Once a fractional solution is computed, randomized rounding is applied to produce an integral solution. Each fractional flow is probabilistically assigned to one of its feasible paths in proportion to its allocation. This ensures that the final solution is deterministic and executable, while still approximating the optimal fractional flow.

Admitted flows are then annotated with their computed paths, shaping parameters, and expected delivery behavior.

4.4 Node Implementation and Packet Flow

Nodes are modeled using custom classes with stateful behavior across simulation cycles.

- IngressNode is responsible for injecting packets into the network according to the computed shaping rate. It reads the schedule for each admitted flow and generates packets only during eligible cycles.
- CoreNode models intermediate forwarding behavior. It accepts packets from incoming links, buffers them based on queuing delay, and forwards them downstream when the cycle and delay budget permit.
- Egress behavior is integrated into the IngressNode or main.py logic, and marks a packet as complete when it reaches its destination.

Each node maintains a cycle-based buffer and tracks flow states such as packet count, delivery success, and any violations of delay constraints.

4.5 Cycle-Based Simulation Engine

The simulation proceeds in discrete time steps, each representing one cycle of fixed duration (e.g., 10 microseconds). In each cycle:

1. Ingress nodes check if packets need to be generated based on their rate and schedule.
2. Core nodes forward packets received in prior cycles if the forwarding slot is active.
3. Packet delay is computed as the sum of queuing and propagation delays across each hop.
4. Successfully delivered packets are logged, and their delivery timestamp is recorded.

The simulation terminates when all admitted flows complete or when a preset timeout is reached. The system gracefully handles keyboard interrupts, saving the partial state when aborted.

4.6 Random Flow Generation

In addition to static configuration, the simulator supports randomized flow generation at runtime. Using command-line flags, users can specify how many flows to generate, optionally overriding the static flow list. Each random flow is created with randomized arrival rate, delay bound, burst size, and source-destination pairs. The topology and timing parameters still derive from the base configuration file.

This feature supports dynamic testing scenarios and allows experimentation with different traffic intensities without editing the JSON manually.

4.7 Output Logging and JSON Reporting

Simulation results are captured in a JSON file with a comprehensive structure. The output includes:

- Simulation parameters (e.g., cycle duration),
- Network topology and delays,
- List of flows, along with their computed paths, shaping parameters, and delivery status,
- Summary statistics such as number of admitted flows, completion status per flow, and total runtime status.

This structured output can be used for further visualization, analysis, or integration with monitoring tools. Intermediate logs also provide real-time visibility into the flow admission and simulation process.

4.8 Summary

This modular, configuration-driven implementation closely mirrors the algorithmic and architectural principles proposed in the source paper [1]. It lays the foundation for future enhancements such as performance benchmarking, topology visualization, or real-time integration with other traffic sources.

Chapter 5

Conclusion and Future Work

THE goal of this project was to design and implement a simulation framework for time-deterministic IP networking, inspired by the algorithmic approach proposed in the research paper “Towards Large-Scale Deterministic IP Networks” [1]. Through this work, I translated the theoretical formulation of the Column Generation with Randomized Rounding (CGRR) algorithm into a modular, extensible Python-based system capable of simulating delay-bounded, cycle-based deterministic packet delivery across an arbitrary network topology.

The implementation offers full support for both static and randomized flow configurations, customizable network parameters, and cycle-driven simulation. It models packet-level behavior with queuing and propagation delays, simulates ingress and core node behaviors, and provides structured JSON output for further analysis. By integrating flow shaping, delay-aware path selection, and randomized rounding into a coherent simulation pipeline, this project provides a strong foundation for further research and experimentation in the domain of deterministic networking.

While the implementation closely follows the algorithmic blueprint outlined in the source paper [1], this project intentionally did not include performance benchmarking or comparative evaluation of simulation results. The primary objective was to ensure the algorithm's correctness and system-level behavior rather than to quantify flow acceptance ratios, delay violations, or scalability characteristics. These directions remain open for future exploration.

5.1 Contributions

This project makes several key contributions by transforming the theoretical principles of deterministic IP networking into a functioning simulation system. Each component has been designed with clarity and modularity in mind, making the system both instructive and extensible. The main contributions are as follows:

- **Implementation of the CGRR Algorithm:** The project delivers a faithful implementation of the Column Generation with Randomized Rounding (CGRR) algorithm as described in the paper [1]. The algorithm computes delay-constrained multi-commodity flows using an LP-based formulation and converts them into deterministic paths via randomized rounding. This forms the decision-making backbone of the simulator, enabling flow admission based on strict timing and resource constraints.
- **Modular, Cycle-Based Simulation Framework:** A fully modular simulation framework was built in Python to model the time-deterministic behavior of networks. The simulator operates on discrete time cycles, reflecting the structure of real-time scheduling systems. Components such as ingress nodes, core nodes, flow shaping, and forwarding logic are encapsulated in dedicated modules, supporting clean separation of concerns and ease of future development.
- **Support for Flexible Configuration:** The system supports configurable input through JSON files, allowing users to define custom topologies, link characteristics, node delays, and flow requirements. Additionally, the simulator can generate

random flows dynamically at runtime, which facilitates broader testing without manual reconfiguration.

- **Structured Output and Logging:** The simulator produces structured output in the form of a JSON report containing all relevant simulation parameters, admitted flows, computed paths, and delivery status. This output format supports further analysis and can be used to validate deterministic scheduling behavior or visualize flow progress over time.
- **Foundation for Future Research:** While the current focus was on implementation rather than evaluation, the simulator lays a strong foundation for future research. It is well-suited for extensions involving performance analysis, topology experimentation, or integration with visualization tools and real-time systems.

These contributions collectively demonstrate the practicality of implementing deterministic networking techniques in a simulated environment and offer a concrete platform for extending the ideas proposed in the original research paper.

5.2 Future Work

While this project successfully implements a deterministic IP network simulation framework based on the CGRR algorithm, there remain several opportunities to enhance, extend, and deepen its capabilities. The following areas represent promising directions for future development:

- **Quantitative Performance Evaluation:** Future work can focus on systematically evaluating the algorithm's performance across various topologies and workloads. Metrics such as flow acceptance rate, end-to-end delay adherence, bandwidth utilization, and simulation runtime can be analyzed to understand trade-offs and scalability limits. This would help validate the simulator's behavior under realistic or stress-test conditions.
- **Visualization of Simulation Behavior:** Introducing graphical interfaces to visualize flow paths, node activity, packet movement, and scheduling cycles would

greatly aid in interpreting and presenting results. A visual dashboard could also help debug flow admission logic and highlight packet-level timing guarantees in real-time.

- **Integration with Real-World Traffic Traces:** Extending the simulator to ingest real-world or synthetic traffic traces could test its applicability to practical networking scenarios. This would also allow researchers to explore how well the CGRR algorithm handles bursty or time-varying flows in deterministic environments.
- **Dynamic and Fault-Tolerant Topologies:** The current system assumes a static topology and fixed set of flows. Future versions could introduce support for dynamic reconfiguration, node or link failures, and flow rerouting. These extensions would simulate more realistic network behaviors and test the robustness of deterministic scheduling under disruptions.
- **Real-Time Emulation or Deployment Hooks:** With minimal adaptation, the simulator could be connected to real packet generators, traffic analyzers, or network emulators. This would allow the tool to function not only as a simulator but also as an emulator for small-scale deterministic network prototypes.
- **Formal Verification and Scheduling Guarantees:** Adding formal analysis techniques could help verify correctness and safety of computed schedules. Such verification could be particularly valuable in safety-critical domains like avionics, manufacturing automation, and autonomous transportation systems.

Bibliography

- [1] B. Liu, S. Ren, C. Wang, V. Angilella, P. Medagliani, S. Martin, and J. Leguay, “Towards large-scale deterministic ip networks,” in *2021 IFIP Networking Conference (IFIP Networking)*, 2021, pp. 1–9.
- [2] N. Finn, P. Thubert, B. Varga, and J. Farkas, “Deterministic networking architecture,” in *RFC 8655*. IETF, 2019.
- [3] E. Grossman, “Deterministic networking use cases,” Tech. Rep., 2019.
- [4] N. Finn, “Introduction to time-sensitive networking,” *IEEE Communications Standards Magazine*, vol. 2, no. 2, pp. 22–28, 2018.

